BÁO CÁO THỰC HÀNH
DỊCH NGƯỢC GAME ANDROID

Phân tích tài nguyên và logic coin của game SandSort

Họ và tên: Nguyễn Đăng Bá Khôi Mã số sinh viên: B24DCGA078

Lớp: D24CQGA02-B

1 Mục tiêu và phạm vi thực hành

Bài thực hành khảo sát cấu trúc dữ liệu của một game Unity Android, xem tài nguyên hình ảnh và lần theo hai luồng xử lý coin. Đối tượng theo đề bài là SandSort; các lớp quan sát được trong quá trình phân tích thuộc namespace FoodSort.

Kết quả xác định được hàm nạp coin khi khởi động, trường lưu coin và luồng cộng 100 coin trong nhánh nhận thưởng của NextLevel. StateCoin cập nhật số dư trong bộ nhớ rồi lưu bằng PlayerPrefs. Đây là kết luận từ phân tích tĩnh; chưa có thử nghiệm thay đổi APK hoặc đo số dư khi game chạy.

2 Môi trường và công cụ

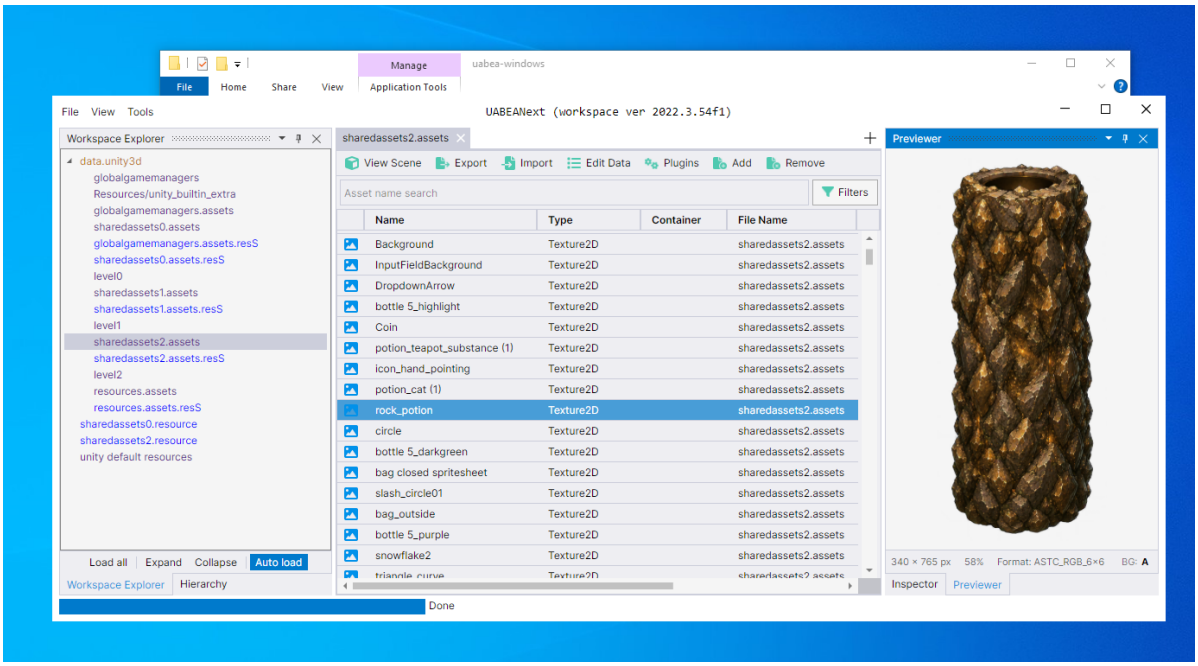
Thành phần	Vai trò và kết quả quan sát
Windows	Giao diện Windows xuất hiện trong các ảnh công cụ và File Explorer.
UABEAnext	Đã mở dữ liệu Unity; workspace hiển thị 2022.3.54f1; xem được Sprite, Texture2D và TextAsset.
Il2CppDumper 6.7.46	Thư mục công cụ có DummyDll, dump.cs, script.json, ghidra.py, stringliteral.json và il2cpp.h.
dnSpy 6.1.8	Đã mở lớp FoodSort.PlayerResourceManager và thuộc tính Coin.
Ghidra	Đã mở libil2cpp.so trong project ThuchanhVM; Listing hiển thị các thanh ghi x0, x1 và lệnh ARM64.

Thư viện native cần dùng thống nhất là lib/arm64-v8a/libil2cpp.so. Các địa chỉ trong báo cáo là địa chỉ hiển thị tại Ghidra hoặc RVA ghi trong metadata; không dùng chúng thay cho File Offset khi đối chiếu tệp.

3 Xem và xuất tài nguyên Unity

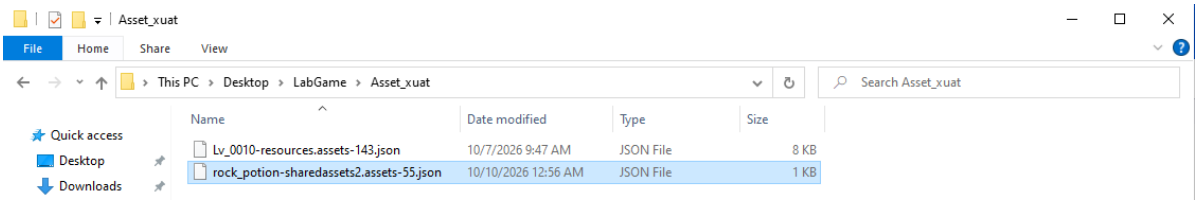
Trong UABEAnext, dữ liệu data.unity3d đã được mở và duyệt đến sharedassets2.assets. Asset được chọn là rock_potion, thuộc loại Texture2D. Preview hiển thị một vật thể dạng bình hoặc ống có bề mặt đá; hình này có thể dùng cho một đối tượng trong màn chơi. Vai trò cụ thể cần đối chiếu với giao diện game đang chạy.

Vị trí dữ liệu Unity theo cấu trúc APK của bài thực hành là assets/bin/Data. File chứa asset đang xem trong workspace là sharedassets2.assets; ảnh Preview hiển thị kích thước 340 × 765 px và định dạng ASTC_RGB_6×6.



Hình 1 Texture2D rock_potion trong sharedassets2.assets và ảnh Preview

Ngoài ra, sharedassets0.assets có Sprite substance_3_color_red; resources.assets có TextAsset Lv_0010 chứa dữ liệu màn chơi. Các kết quả này cho thấy tài nguyên hình ảnh và dữ liệu level được lưu thành những loại asset khác nhau.



Hình 2 Các tệp đã xuất trong Desktop\LabGame\Asset_xuat

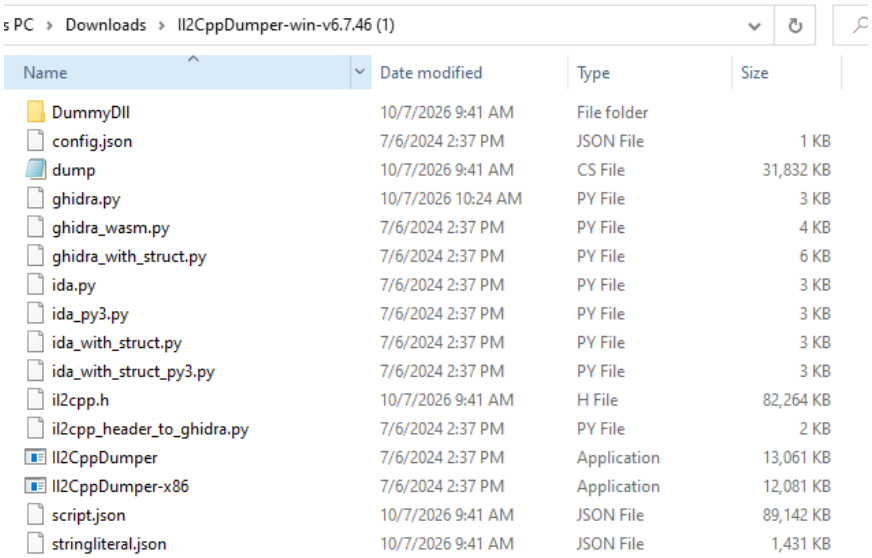
Thư mục xuất có rock_potion-sharedassets2.assets-55.json và Lv_0010-resources.assets-143.json. Đây là dữ liệu JSON đã xuất; tệp JSON của Texture2D không thay thế cho ảnh PNG dùng làm minh chứng xuất tài nguyên hình ảnh. Bản nộp cần kèm ảnh rock_potion đã xuất bằng chức năng xuất texture của công cụ.

4 Tra cứu cấu trúc và địa chỉ IL2CPP

4 1 Dữ liệu đầu vào và kết quả dump

Trước khi mở và phân tích libil2cpp.so trong Ghidra thì cần phải sửa lại file ghidra.py do lỗi khiến python không thể đọc được.

Cặp dữ liệu đầu vào theo bài thực hành gồm lib/arm64-v8a/libil2cpp.so và assets/bin/Data/Managed/Metadata/global-metadata.dat của cùng APK. Il2CppDumper kết hợp mã native với metadata để tạo dữ liệu tra cứu. Cần giữ hai file đầu vào và các kết quả của cùng lần dump để đối chiếu.



Hình 3 Thư mục Il2CppDumper 6.7.46 có các đầu ra phục vụ phân tích

Thư mục Downloads/Il2CppDumper-win-v6.7.46 (1) hiển thị DummyDll, tệp dump loại CS File, script.json, ghidra.py, stringliteral.json và il2cpp.h. Kết hợp với ảnh mở dump.cs và lớp trong dnSpy, đây là bằng chứng về dữ liệu tra cứu đã tạo.

Đầu ra	Cách sử dụng
dump.cs	Tra cứu namespace, lớp, offset trường và RVA của phương thức.
DummyDll	Mở DLL trong dnSpy để xem cấu trúc lớp và chữ ký phương thức.
script.json và ghidra.py	Gắn tên phương thức vào vị trí mã native trong Ghidra.
stringliteral.json	Đối chiếu nội dung chuỗi với các ký hiệu StringLiteral khi cần xác định khóa dữ liệu.

Dữ liệu dump hỗ trợ tìm ứng viên. Kết luận về hành vi được kiểm chứng bằng mã native trong Listing và Decompiler; các thân hàm rỗng của DLL giả không được dùng để suy ra logic chạy thực tế.

4 2 Xác định lớp và trường coin

```
using System;
using Il2CppDummyDll;

namespace FoodSort
{
    // Token: 0x02000186 RID: 390
    [Token(Token = "0x2000186")]
    public class PlayerResourceManager : Singleton<PlayerResourceManager>
    {
        // Token: 0x170000EB RID: 235
        // (get) Token: 0x06000A38 RID: 2616 RVA: 0x00004BC0 File Offset: 0x00002DC0
        // (set) Token: 0x06000A39 RID: 2617 RVA: 0x00002053 File Offset: 0x00000253
        [Token(Token = "0x170000EB")]
        public int Coin
        {
            [Token(Token = "0x6000A38")]
            [Address(RVA = "0x22A0708", Offset = "0x229C708", VA = "0x22A0708")]
            get
            {
                return 0;
            }
            [Token(Token = "0x6000A39")]
            [Address(RVA = "0x22A0710", Offset = "0x229C710", VA = "0x22A0710")]
            set
            {
            }
        }
```

Hình 4 Thuộc tính Coin trong lớp PlayerResourceManager ở dnSpy

Ảnh dnSpy hiển thị Coin có getter RVA 0x022A0708 và setter RVA 0x022A0710. Getter trong DLL giả trả về 0 và setter có thân rỗng; đây là phần dựng lại cấu trúc, không phải bằng chứng rằng game luôn có 0 coin.

```
// Namespace: FoodSort
public class PlayerResourceManager : Singleton<PlayerResourceManager> // TypeDefIndex: 6336
{
        // Fields
        public static Action<int> OnCoinChange; // 0x0
        public static Action<int> OnBoosterUndoChange; // 0x8
        public static Action<int> OnBoosterExtraChange; // 0x10
        public static Action<int> OnBoosterShuffleChange; // 0x18
        private int _coin; // 0x20
        private int _numOfBoosterUndo; // 0x24
        private int _numOfBoosterExtra; // 0x28
        private int _numOfBoosterShuffle; // 0x2C

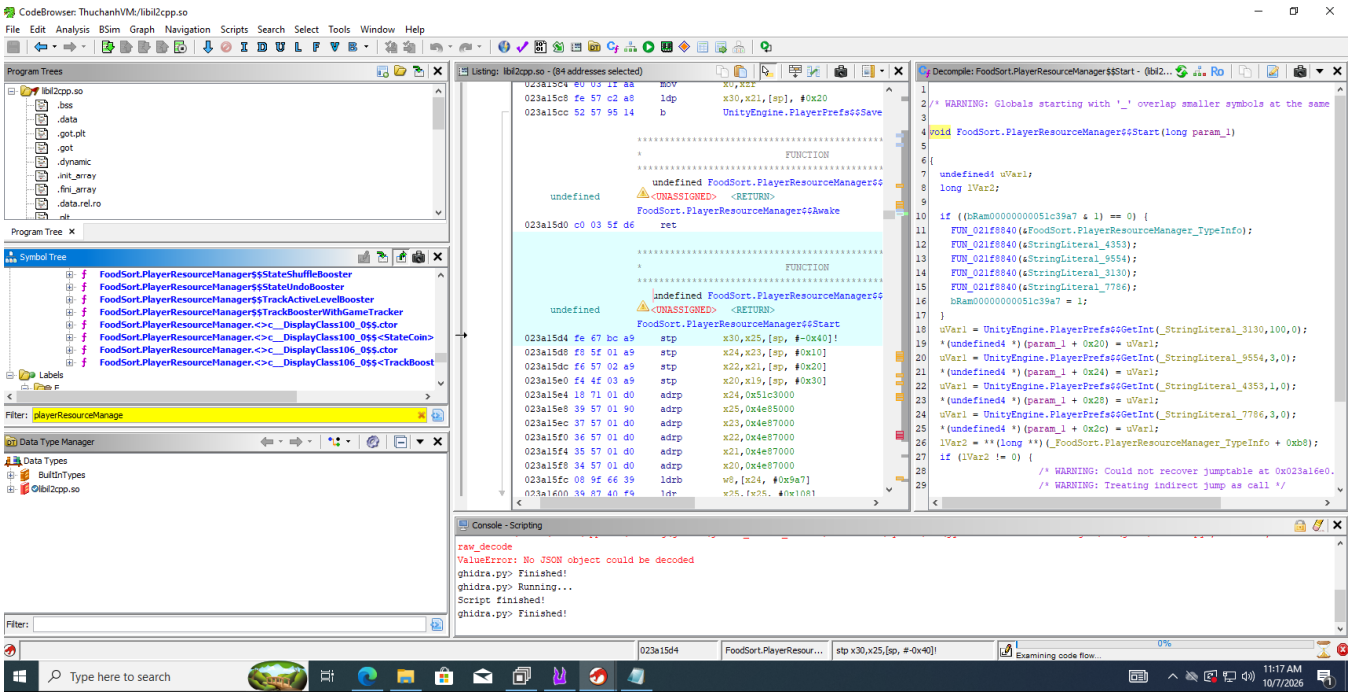
        // Properties
        public int Coin { get; set; }
        public int NumOfBoosterUndo { get; }
```

Hình 5 dump.cs xác định trường _coin tại offset 0x20

Lớp PlayerResourceManager kế thừa Singleton<PlayerResourceManager>. Trường _coin có kiểu int và offset 0x20; ba trường booster kế tiếp có offset 0x24, 0x28 và 0x2C. Offset trường mô tả vị trí dữ liệu trong đối tượng, khác với địa chỉ bắt đầu của một hàm.

5 Logic nạp coin khi vào game

Hàm xử lý được xác định là FoodSort.PlayerResourceManager.Start, có địa chỉ bắt đầu 0x023A15D4 trong Ghidra. Decompiler cho thấy hàm đọc giá trị từ PlayerPrefs rồi ghi trực tiếp vào đối tượng PlayerResourceManager.



Hình 6 Listing và Decompiler của PlayerResourceManager.Start

```
uVar1 = UnityEngine.PlayerPrefs$$GetInt(
    _StringLiteral_3130, 100, 0);
*(undefined4 *)(param_1 + 0x20) = uVar1;
```

Tham số 100 là giá trị mặc định của GetInt khi khóa tương ứng chưa tồn tại. Nếu đã có dữ liệu ở khóa này, hàm lấy giá trị đã lưu. Vì vậy, kết quả không có nghĩa mỗi lần vào game đều đặt coin về 100.

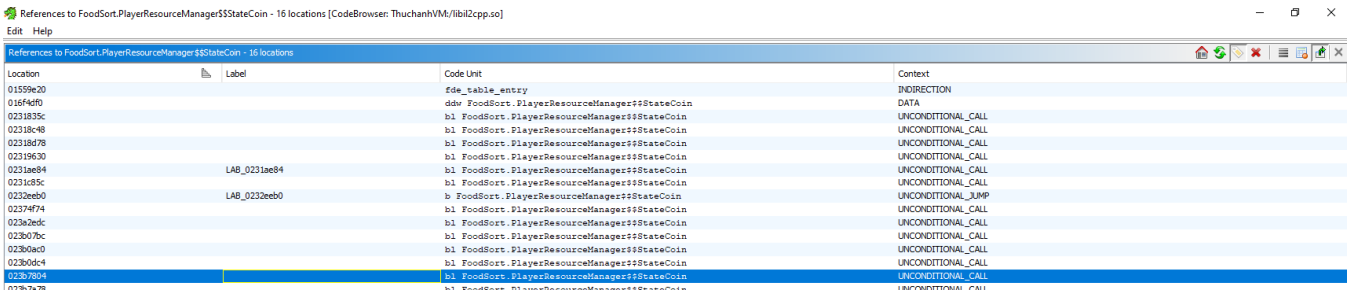
Giá trị đọc về được ghi vào param_1 + 0x20. Đối chiếu với Hình 5, vị trí này chính là trường _coin. Đây là bằng chứng nối nguồn dữ liệu trong PlayerPrefs với nơi lưu số coin đang dùng trong bộ nhớ.

Ba lời gọi GetInt tiếp theo có mặc định 3, 1 và 3, rồi ghi vào các offset 0x24, 0x28 và 0x2C. Chúng khớp với các trường booster; không dùng các giá trị này làm số coin ban đầu. _StringLiteral_3130 là tên ký hiệu do công cụ gắn, chưa phải nội dung chuỗi khóa đọc được.

6 Logic coin trong luồng nhận thưởng

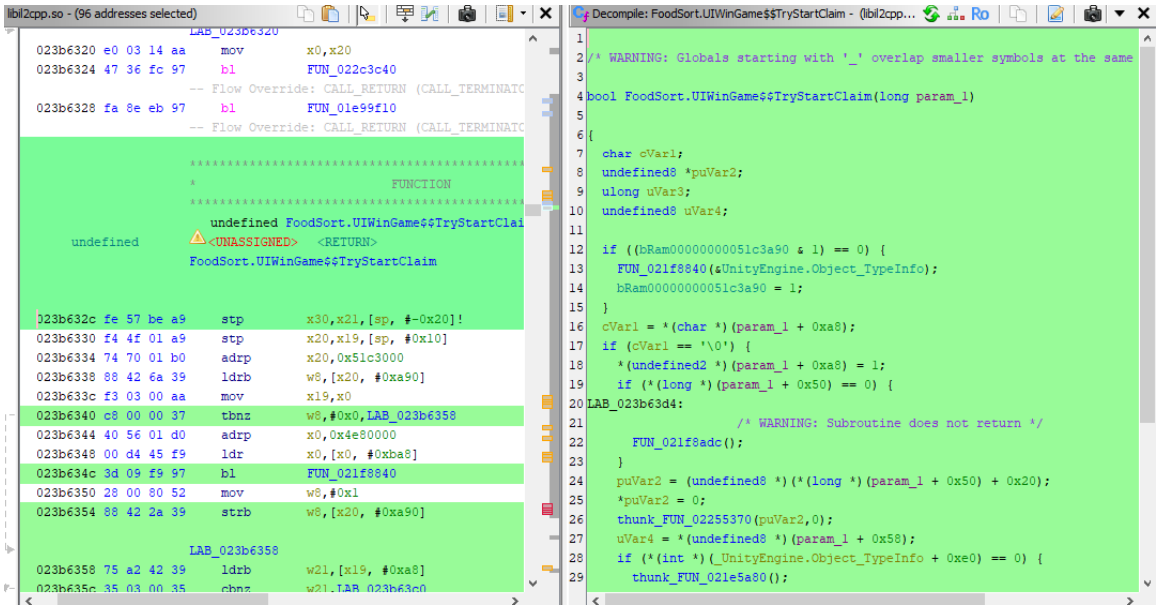
6 1 Xác định các vị trí gọi xử lý coin

Từ PlayerResourceManager.StateCoin, cửa sổ References của Ghidra liệt kê 16 vị trí tham chiếu. Danh sách bao gồm tham chiếu dữ liệu và các lệnh gọi; vì vậy số 16 không đồng nghĩa có 16 nhánh thưởng level. Vị trí lệnh gọi đang được chọn trong ảnh là 0x023B7804.



Hình 7 Danh sách tham chiếu đến PlayerResourceManager.StateCoin

Luồng nhận thưởng liên quan đến lớp UIWinGame. Hàm FoodSort.UIWinGame.TryStartClaim có địa chỉ bắt đầu 0x023B632C. Phần đầu của hàm đọc cờ tại param_1 + 0xA8; ở một nhánh, hàm ghi 1 vào cờ trước khi xử lý các đối tượng giao diện tiếp theo.

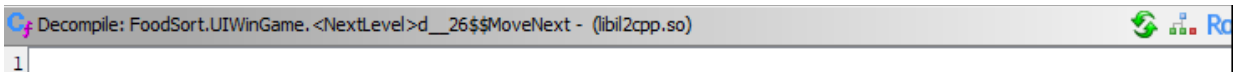


Hình 8 TryStartClaim kiểm tra cờ trạng thái trong UIWinGame

TryStartClaim là ứng viên kiểm soát trạng thái nhận thưởng. Phần mã đang thấy chưa đủ để khẳng định toàn bộ cơ chế ngăn nhận thưởng lặp hoặc thời điểm đặt lại cờ. Hàm này không được coi là hàm trực tiếp cộng vào trường _coin.

6 2 Nhánh nhận thưởng trong NextLevel

FoodSort.UIWinGame.<NextLevel>d__26.MoveNext thực thi phần xử lý bất đồng bộ của NextLevel. Ở nhánh bắt đầu, hàm gọi TryStartClaim. Nếu bit thấp của kết quả bằng 0, luồng nhảy đến LAB_023B78C0 để kết thúc; nếu được phép tiếp tục, hàm lấy đối tượng PlayerResourceManager rồi gọi StateCoin.



Tên hàm MoveNext hiển thị trong Ghidra

```
33      }
34      uVar1 = FoodSort.UIWinGame$$TryStartClaim();
35      if ((uVar1 & 1) == 0) goto LAB_023b78c0;
36      lVar2 = FoodSort.Singleton<object>$$get_Instance
37                        (_Method$FoodSort.Singleton<PlayerResourceManager>.get_Instance());
38      if (lVar2 == 0) {
39                        /* WARNING: Subroutine does not return */
40        FUN_021f8adc();
41      }
42      FoodSort.PlayerResourceManager$$StateCoin
43                (lVar2,1,100,_StringLiteral_13854,_StringLiteral_14787,0);
44      if (*(int *)(_System.Threading.Tasks.Task_TypeInfo + 0xe0) == 0) {
45        thunk_FUN_021e5a80();
46      }
47      lVar2 = System.Threading.Tasks.Task$$Delay(300,0);
48      if (lVar2 == 0) {
```

Hình 9 TryStartClaim và lời gọi StateCoin với cờ 1 cùng giá trị 100

```
FoodSort.PlayerResourceManager$$StateCoin(
   lVar2, 1, 100,
   _StringLiteral_13854,  _StringLiteral_14787,  0);
```

Thân StateCoin ở mục 6 3 xác nhận cờ 1 chọn dấu dương. Do đó, lời gọi này cộng 100 vào số dư hiện tại. Kết luận áp dụng cho nhánh NextLevel đang khảo sát; không suy rộng thành mọi loại phần thưởng trong game đều là 100 coin. Nội dung hai chuỗi _StringLiteral_13854 và _StringLiteral_14787 chưa được giải mã.

```
65    System.Runtime.CompilerServices.TaskAwaiter$$GetResult(&uStack_18,0);
66    lVar2 = FoodSort.Singleton<object>$$get_Instance
67                      (_Method$FoodSort.Singleton<LevelManager>.get_Instance());
68    if (lVar2 == 0) {
69                      /* WARNING: Subroutine does not return */
70      FUN_021f8adc();
71    }
72    FoodSort.LevelManager$$ContinueAfterWin(lVar2,0);
73  LAB_023b78c0:
74    *param_1 = -2;
75    System.Runtime.CompilerServices.AsyncVoidMethodBuilder$$SetResult(param_1 + 2,0);
76    return;
77  }
```

Hình 10 Sau khi chờ tác vụ hàm gọi ContinueAfterWin rồi hoàn tất

Sau lời gọi StateCoin, mã tạo Task.Delay(300), xử lý trạng thái chờ rồi gọi LevelManager.ContinueAfterWin. Như vậy, thứ tự ở nhánh này là kiểm tra nhận thưởng, cập nhật và lưu coin, chờ tác vụ, sau đó tiếp tục sau màn thắng. Nhãn LAB_023B78C0 dùng để hoàn tất phương thức.

6 3 Cộng coin và lưu số dư trong StateCoin

FoodSort.PlayerResourceManager.StateCoin nhận đối tượng quản lý tài nguyên, cờ chọn dấu và lượng coin cần xử lý. Phần đầu mã sao chép param_2 & 1 vào offset 0x10 của đối tượng hỗ trợ và param_3 vào offset 0x14. Đây là dữ liệu tạm cho thao tác cập nhật, khác với trường _coin của PlayerResourceManager.

```
}
lVar2 = thunk_FUN_02203abc(_FoodSort.PlayerResourceManager.<>c__DisplayClass100_0_TypeInfo);
System.Object$$.ctor(lVar2,0);
if (lVar2 != 0) {
  *(undefined8 *)(lVar2 + 0x18) = param_4;
  *(byte *)(lVar2 + 0x10) = param_2 & 1;
  *(undefined4 *)(lVar2 + 0x14) = param_3;
```

Hình 11 StateCoin lưu cờ xử lý và lượng coin vào đối tượng hỗ trợ

```
thunk_FUN_02255370((undefined8 *)(lVar2 + 0x18),param_4);
if (param_1 != 0) {
  iVar1 = -*(int *)(lVar2 + 0x14);
  if (*(char *)(lVar2 + 0x10) != '\0') {
    iVar1 = *(int *)(lVar2 + 0x14);
  }
  iVar1 = iVar1 + *(int *)(param_1 + 0x20);
  *(int *)(param_1 + 0x20) = iVar1;
  UnityEngine.PlayerPrefs$$SetInt(_StringLiteral_3130,iVar1,0);
  UnityEngine.PlayerPrefs$$Save(0);
  puVar7 = (undefined8 *)(lVar2 + 0x20);
```

Hình 12 Phép cộng vào _coin và các lời gọi SetInt cùng Save

```
delta = -amount;
if (flag != 0) delta = amount;
newCoin = oldCoin + delta;
PlayerPrefs.SetInt(coinKey, newCoin);
PlayerPrefs.Save();
```

Đoạn mã trên diễn giải các lệnh trong Hình 12. Khi cờ bằng 0, lượng thay đổi mang dấu âm; khi cờ bằng 1, lượng thay đổi mang dấu dương. Mã lấy số dư ở param_1 + 0x20, cộng lượng thay đổi rồi ghi trở lại đúng vị trí này. Với lời gọi ở Hình 9, số dư mới bằng số dư cũ cộng 100.

Ngay sau khi ghi _coin, hàm gọi PlayerPrefs.SetInt(_StringLiteral_3130, iVar1, 0) và PlayerPrefs.Save(). Ký hiệu khóa _StringLiteral_3130 trùng với khóa đọc trong Start. Vì vậy, luồng đọc lúc khởi động và luồng lưu sau cập nhật cùng xử lý một dữ liệu coin.

```
    FoodSort.GameManager$$TrackWhenMetadataReady(uVar5,0);
    lVar2 = **(long **)(_FoodSort.PlayerResourceManager_TypeInfo + 0xb8);
    if (lVar2 != 0) {
      (**(code **)(lVar2 + 0x18))
                (*(undefined8 *)(lVar2 + 0x40),*(undefined4 *)(param_1 + 0x20),
                 *(undefined8 *)(lVar2 + 0x28));
    }
    return;
  }
}
```

Hình 13 StateCoin gọi phần theo dõi và callback sau khi cập nhật

Sau cập nhật, hàm có RecordLevelCoinChange, TrackWhenMetadataReady và lời gọi gián tiếp truyền số dư. Cần đối chiếu callback để kết luận cách cập nhật giao diện. Địa chỉ 0x023B7804 là vị trí gọi StateCoin, không phải địa chỉ bắt đầu của hàm.

7 Nhận xét về an toàn tài nguyên phía client

Luồng khởi động sử dụng dữ liệu lưu cục bộ để nạp _coin. Luồng nhận thưởng truyền một giá trị vào hàm xử lý tài nguyên trên client. Đây là các vị trí cần đánh giá khi xem xét khả năng can thiệp vào tài nguyên của game.

Một kịch bản cần kiểm chứng là giá trị phần thưởng phía client bị thay đổi trước khi chuyển vào hàm cập nhật coin. Nếu game chấp nhận giá trị này mà không có kiểm tra độc lập, tổng coin có thể sai lệch, làm người chơi mua vật phẩm hoặc sử dụng tài nguyên ngoài tiến trình dự kiến. Tương tự, dữ liệu coin lưu cục bộ cần được kiểm tra tính hợp lệ khi nạp.

Phân tích tĩnh xác định vị trí và luồng dữ liệu, nhưng không chứng minh việc can thiệp đã thành công. Cần kiểm tra thêm hành vi khi game chạy, điều kiện nhận thưởng, thời điểm lưu và khả năng đồng bộ hoặc đối chiếu với máy chủ. Báo cáo không khẳng định game có hay không có cơ chế xác thực trên máy chủ.

Với game có dịch vụ trực tuyến, một phương án giảm phụ thuộc vào client là để máy chủ xác nhận kết quả level, tính phần thưởng theo quy tắc đã lưu và cập nhật số dư. Client gửi kết quả cần xác minh và nhận số dư đã xác nhận. Mỗi lượt nhận thưởng cần có mã định danh để kiểm tra việc xử lý trùng. Đây là hướng đề xuất; chưa phải mô tả cơ chế đang tồn tại trong APK.

8 Kết quả và kết luận

Hạng mục	Kết quả phân tích
Tài nguyên	Đã xem rock_potion loại Texture2D trong sharedassets2.assets; minh chứng xuất hiện có là JSON.
Coin khi khởi động	Start tại 0x023A15D4 đọc PlayerPrefs với mặc định 100 và gán vào _coin ở offset 0x20.
Nhận thưởng	NextLevel.MoveNext kiểm tra TryStartClaim rồi gọi StateCoin với cờ 1 và lượng 100; nhánh này cộng 100 coin.
Cập nhật tổng coin	StateCoin ghi số dư mới vào _coin ở offset 0x20, sau đó gọi PlayerPrefs.SetInt và PlayerPrefs.Save.
Khả năng can thiệp	Nêu được kịch bản liên quan đến tài nguyên phía client; chưa có kết quả thử nghiệm động.

Qua bài thực hành, em xác định được cách chuyển từ tên lớp và offset trong dữ liệu dump sang mã native trong Ghidra. Luồng nạp coin có bằng chứng trực tiếp về nguồn giá trị và trường lưu; luồng thưởng xác định được điều kiện gọi, phép cộng 100 và thao tác lưu số dư. Khi đánh giá an toàn, cần phân biệt nhận định từ mã với kết quả được kiểm chứng khi chạy game.

Tài liệu đối chiếu: đề LAB Kỹ thuật dịch ngược game Android, ảnh trong Báo cáo an toàn game và các ảnh bổ sung về Il2CppDumper, StateCoin, NextLevel.MoveNext. Hình 1 đến Hình 13 sử dụng ảnh thực hành hoặc phần phóng lớn từ các ảnh này.